

When Does LLM-Serving Scheduler Adaptation Matter? Action Opportunity and Causal Headroom in Production-Derived Replay

Soroush Vahidi*

*Department of Computer Science, Ying Wu College of Computing,
New Jersey Institute of Technology, University Heights,
Newark, NJ 07102, USA*

Abstract

Adaptive scheduling for large language model (LLM) serving pays off only if the system reaches states with a different executable action that improves performance. We measure that opportunity before designing any adaptive scheduler. In a deterministic simulator replaying production-derived traces (Azure code, Azure conversation, BurstGPT), we count how often six policies propose an action different from a fixed reference and, on fresh windows, force one alternative for one step. With abundant resources, none of about one million decision states offered a different executable action. Capping key-value (KV) cache capacity or concurrent sequences created such states; scaling arrival rates up to eight times left the system lightly loaded and created none. This held on fresh windows for the Azure traces but not BurstGPT, which never queued enough for the caps to bind. In a pre-specified analysis of 720 fresh disagreement states, 590 (81.9%) had positive latency headroom (state-weighted mean simulated headroom 2.00 ms; 95% window-clustered confidence interval 0.19–3.55 ms). The sign is stable; the magnitude is not: the median is 0.20 ms, one of 36 windows holds 88% of the total, and in 14% of states every alternative is worse. About 95% of the headroom lies where all five other policies differ from the reference. A bounded vLLM probe shows

*Corresponding author.

Email address: `sv96@njit.edu` (Soroush Vahidi)

that pressure changes scheduler behavior but does not validate the simulated latency effect. The opportunity is real but sparse, concentrated, and reference-conditional; finding it does not show that adaptation is worthwhile.

Keywords: LLM serving, request scheduling, performance evaluation, workload replay, resource contention, causal evaluation

1. Introduction

Adaptive scheduling in large language model (LLM) serving is usually motivated by heterogeneous prompts, decode demand, key-value (KV) cache residency, queueing, and deadlines. It can help only if the live system reaches states where a different executable action exists, such as another admission, batch, or request order, and only if taking that action improves the target metric by more than the cost of choosing it. How often this happens is not usually reported alongside a scheduler design.

Common evaluation practice does not answer this. Ranking policies on whole-trace metrics, or reporting the gap to a virtual best solver (VBS) as in algorithm portfolios [1, 2], shows that policies differ but aggregates thousands of decisions; it cannot say whether an alternative was available at a given state. Policies with different scores can issue the same batch, and an alternative that exists may not reduce latency. Recent systems propose mechanisms that adapt scheduling and routing to request, cache, and load state [3, 4, 5], and others optimize serving under request and SLO state [6, 7, 8]. We ask the prior question: *when is there a different useful action to take, and how much is it worth?*

We answer it with a staged decomposition,

$$\begin{aligned} \text{resource pressure} &\longrightarrow P(D) \longrightarrow P(B_{\text{LAT}} \mid D) \\ &\longrightarrow \text{headroom magnitude} \\ &\longrightarrow \text{predictability and deployment.} \end{aligned} \tag{1}$$

Here D marks a *disagreement state*, one in which at least one canonical executable alternative (an admission, batch, and ordering decision after infeasible or identical variants are collapsed; Section 3.3) differs from the default single best scheduler (SBS), a fixed reference policy (Section 3.2), and B_{LAT} marks a state in which at least one such alternative lowers one-step mean latency relative to that reference. These are distinct quantities: pressure

can make policies disagree often without the disagreement being useful, and useful disagreement can be rare or small. All of them are conditional on the reference and the portfolio. The last arrow, whether a selector could exploit the opportunity, is not measured here (Figure 1).

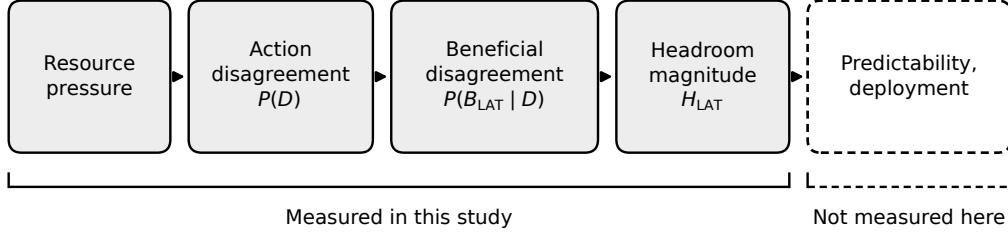


Figure 1: Staged evidence chain (Eq. 1): resource pressure, canonical action disagreement $P(D)$, beneficial disagreement $P(B_{\text{LAT}} | D)$, headroom magnitude H_{LAT} , and predictability and deployment. The study measures the first four stages (solid boxes); the fifth (dashed box) is not measured.

This is a performance-evaluation question: at which operating points does scheduler choice have causal performance consequences? Answering it takes workload characterization, controlled resource interventions, replay-based simulation, and inference that respects the dependence among states from one trace window. It complements recent work in *Performance Evaluation* on assignment, dispatching, and prediction mechanisms (Section 2).

We replay production-derived traces (Azure 2023 code and conversation [9], BurstGPT [10]) in a deterministic simulator. Faithful replay of 60 windows gives the native baseline; one-axis interventions on active-sequence capacity, KV capacity, and arrival rate (1,080 conditions) locate where disagreement appears. We repeat that matrix on fresh non-overlapping windows and, at each disagreement state in five selected pressure regimes (Section 3.1), clone the simulator, force one alternative action once, and return to the reference. The primary endpoint, oracle latency headroom (the largest one-step mean-latency reduction among the alternatives, zero if none helps), was pre-specified before any outcome was computed (Section 3.4); the robustness analyses are post hoc. A bounded vLLM [11] probe checks whether the pressure mechanism appears in a real engine. The oracle is an opportunity bound, not a scheduler.

The study addresses five research questions, ordered along the decomposition.

- RQ1 – Native opportunity.** How often does the SBS face a different executable action from the six-policy portfolio in faithful replay with abundant resources?
- RQ2 – Pressure-induced opportunity.** Which of the tested capacity caps and arrival rates create action disagreement, and does the transition reproduce on fresh windows in every workload?
- RQ3 – Causal value.** At disagreement states, how often does forcing one alternative action for one step reduce latency?
- RQ4 – Magnitude and concentration.** How large and how concentrated is the headroom, and how sensitive is it to weighting, to individual windows or regimes, and to the reference policy?
- RQ5 – System correspondence.** Does a bounded vLLM probe show the corresponding pressure and scheduling mechanism?

1.1. Contributions

This paper proposes no new scheduler. The scheduling heuristics, replay simulation, KV-capacity constraints, and bootstrap inference it uses are established components; the contribution is a measurement built from them and its application.

1. A staged action-opportunity evaluation that separates executable disagreement, beneficial disagreement, and headroom magnitude from policy-score diversity, with prevalence measured over all valid decision states, uncertainty quantified by trace window, and every quantity stated relative to a fixed reference scheduler.
2. A characterization of where disagreement appears in production-derived replays: which resource interventions create it, and whether the pattern reproduces on fresh windows and across workloads.
3. A pre-specified one-step causal measurement on 720 fresh states, with latency as the primary endpoint because deadline goodput saturates, and with harm structure and relative headroom as secondary outcomes.
4. An analysis of how the measured headroom depends on aggregation, individual windows and regimes, and the reference policy.
5. Bounded system evidence from vLLM that resource pressure changes scheduling behavior, without validating simulated latency magnitudes or a deployed selector.

Section 2 positions the work, and Section 3 defines the design, estimands, and endpoints. Sections 4 to 7 report the measurements, and Sections 8 to 9 interpret, state limitations, and conclude.

2. Related Work and Positioning

Serving systems. Iteration-level scheduling and paged KV-cache management underlie current engines [12, 11], and later systems change how prefill and decode are batched, separated, or preempted [13, 14, 15, 16]. Across instances, Llumnix migrates requests at run time, Preble and LMetric route requests using KV-reuse and load indicators, and Libra partitions requests into micro-requests [3, 4, 5, 17]. Within an engine, SOLA schedules with awareness of request-level and system-level state, JITServe allocates bandwidth just in time under imprecise request information, and AdaServe uses SLO-customized speculative decoding for multi-SLO serving [6, 7, 8]. Mooncake, Strata, and MorphServe address KV capacity through a KV-centric architecture, hierarchical context caching, and run-time cache resizing [18, 19, 20]. Three recent mechanisms tie the scheduling decision to memory or load pressure: Apt-Serve changes batch composition over a hybrid KV and hidden-state cache under GPU-memory limits, PKAS regulates admission using predicted future KV-cache demand, and P-PAS adapts vLLM’s batched-token budget to concurrent prefill and decode pressure [21, 22, 23]. DynamoLLM and OpenTela operate at cluster scale, reconfiguring inference clusters to reduce energy and cost under latency objectives and unifying decentralized computing resources for LLM serving [24, 25].

Workloads. BurstGPT and the Azure traces are the production-derived workloads replayed here [10, 9]. ServeGen characterizes production LLM workloads and generates new ones [26], and Nixon et al. study workload evolution, caching, and load balancing over a year of production traffic [27].

Analysis and simulation. Analytical work studies online scheduling under a KV-cache memory limit, with competitive-ratio analysis and a hindsight-optimal benchmark [28]; throughput-optimal and SLO-aware scheduler design for two-phase LLM inference [29]; and the use of service-time predictions in queueing-based scheduling, with LLM serving as a central application [30]. Beyond Prediction studies tail-aware scheduling that is robust to prediction error, burstiness, and memory pressure [31]; this study takes mean latency as its primary endpoint and gives several policies exact output lengths, so it does not evaluate that problem. Simulators such as Vidur predict latency

from profiled operator costs [32]; the simulator used here is not calibrated in that way (Section 3.1).

The question here is upstream of policy design. The systems above design mechanisms that exploit state or resource dependence, and the analytical works analyze or optimize scheduling policies; Jaillet et al. are especially close on KV-cache-constrained online scheduling and hindsight comparison, and Bari et al. on throughput-optimal, SLO-aware scheduler design. This study instead measures, in a fixed workload and resource regime, whether a fixed reference scheduler and its alternatives expose different executable actions at all, and whether forcing one alternative has reference-relative causal value; the measurement is empirical, one-step, and tied to the states a specific reference visits in replay. Recent papers in *Performance Evaluation* address neighboring problems: job assignment in machine-learning inference systems with accuracy constraints [33], dispatching policies in data-center clusters studied on Google and Alibaba workloads [34], and resource-aware latency prediction for GPU-accelerated model serving [35]. Table 1 summarizes the systems closest to this measurement.

The notions of a single best solver and a virtual best solver come from algorithm portfolios and selection [1, 2, 36, 37]. Here the SBS is a fixed default whose executable actions are compared with those of the other policies, and the oracle headroom of Section 3.3 plays the role of the virtual best solver at the level of one-step actions rather than whole-trace scores.

3. Methods

3.1. Workloads and simulator

Native replay of unmodified windows from production-derived traces answers RQ1. One-axis transformations of those windows (caps on active sequences or KV capacity, and arrival-rate scaling) answer RQ2, first on the original windows, which were used to develop the pressure map, and again on fresh windows. A pre-specified one-step intervention on the fresh windows answers RQ3 and RQ4, and the vLLM probe answers RQ5.

The faithful corpus contains 20 windows of 200 requests from each of Azure 2023 code, Azure 2023 conversation, and BurstGPT, with source order and token-length and arrival structure retained. Every replay runs on one simulated GPU (512 active-sequence slots, a 512-token batch budget, 8,000,000 KV tokens by default) with a 1 ms step. The simulator clock has a physical-time interpretation (arrivals and deadlines are in seconds), but

Table 1: Systems closest to this study, by focus and the level at which they act, and their relation to the present measurement.

Work	Focus	Level	Relation to this study
ServeGen [26]	Characterization and generation of production LLM workloads	Workload model	Complements the replay traces used here; measures no scheduling action
LMetric, Preble [5, 4]	KV-aware and load-aware request routing	Across instances	Mechanisms whose routing states this measurement could examine
Llumnix [3]	Cross-instance request rescheduling and migration	Across instances	Adaptive mechanism whose opportunity this measurement targets
SOLA, JITServe, AdaServe [6, 7, 8]	SLO-aware or state-aware scheduling and decoding	Within an engine or request	This study uses loose uniform SLOs and measures action opportunity rather than SLO-system gain
Mooncake, Strata [18, 19]	KV-centric architecture and scheduling; hierarchical context caching	KV placement, scheduling	KV capacity is also a pressure axis studied here
Jaillet et al. [28]	Online scheduling under a KV-cache memory limit, with competitive-ratio analysis	Within an engine	Analytical counterpart of the KV axis; hindsight benchmark
This study	Existence and one-step counterfactual value of alternative actions	Within one engine, per step	—

the step is nominal, so the absolute latencies reported for the replay experiments are simulator-time milliseconds and are not calibrated to measured hardware service times. GPU-calibrated service curves available elsewhere in the project were not used in these experiments. Each request carries a loose service-level objective (SLO) deadline of arrival plus 1000 s, a uniform priority of 1.0, and a single class, and policies observe a predicted output length equal to the true length. No request therefore approaches its deadline (the largest mean continuation latency in any causal population is 0.36 s), and deadline- and priority-sensitive rules receive no informative signal.

A second set of 20 *fresh* windows per workload, taken as the first unused

contiguous windows in trace order and sharing no source records with the first, tests whether the pressure transition reproduces and supplies the states for the causal intervention. Those states come from five pressure *regimes*, each a workload, pressure axis, and pressure setting pooled over windows, chosen mechanically from the support map of the fresh windows (the onset, sustained, and strongest valid setting on each axis that showed disagreement) without using any latency outcome. A pressure condition is a workload window plus one fixed one-axis intervention, such as a KV-capacity cap, active-sequence cap, or arrival-rate scaling factor. A *decision state* is one invocation of the reference policy.

3.2. Scheduling policies and reference

Each policy decides which waiting requests to admit to a GPU’s running batch and in what order. The portfolio contains six policies implemented in the simulator: full prefill and small-chunk prefill (64-token chunks), which share one arrival-order admission rule and differ only in execution chunk size; estimated-service-first (ESTF), which admits the smallest estimated service time (prompt length plus predicted output length); weighted fair sharing (WFS), which ranks by class share and estimated service; least-laxity first (LLF), which ranks by slack to the deadline; and the KV-constrained online policy described below. They are simple heuristics named after standard scheduling rules, not reimplementations of published serving systems.

All six read the same observable state: each waiting request’s arrival time, prompt length, predicted output length, deadline, priority, and class, and each GPU’s capacity and KV occupancy. Because the predicted output length equals the true length, the service-time estimates are exact. Four policies use it (ESTF, WFS, LLF, and the reference); the two prefill variants use only arrival order. The rules are therefore oracle-informed heuristics rather than deployable schedulers, and the results describe this idealized information setting. The portfolio is also functionally less diverse than six names suggest. The two prefill variants issue the same action from any state. With one class and uniform priority, WFS’s class-share term is constant, and in the 720 causal states it differs from the reference in exactly the same 519 states as ESTF. LLF ranks by slack to deadlines 1000 s away. At the 720 disagreement states the portfolio offers on average 1.15 distinct alternative actions, and 610 states have exactly one.

The reference (default) scheduler is the single best scheduler (SBS), the KV-constrained online policy. It had the highest mean goodput among the

six policies on a separate earlier benchmark of 240 multi-mechanism scenarios that used the same simulator and policy implementations but different trace windows. It was fixed before the fresh evaluation and was not selected, tuned, or changed using fresh outcomes, these workloads, or latency. It is best only among these six policies on that benchmark and is not claimed to be optimal or strong here. It orders waiting requests by KV footprint and admits a request only if the resulting KV utilization stays at or below a reserve of 0.82 of the configured capacity, unless the request is urgent (slack under 0.25 s), an exemption that never applies with 1000 s deadlines. Every measurement below is therefore relative to this reference and its reserve. Three notions of KV pressure are kept apart: the imposed capacity setting (the cap value); physical binding, when the waiting prompt tokens exceed the free KV capacity (for the active-sequence cap, when no sequence slot is free); and the reference’s own reserve threshold, which can withhold admission before capacity is physically exhausted.

3.3. Action opportunity and one-step headroom

Let $\mathcal{A}_6(s)$ be the set of canonical executable actions produced by the six portfolio policies at state s , after action deduplication. A canonical action is the simulator-executable admission, batch, and request ordering decision after infeasible or semantically identical variants have been collapsed. Let $a_{\text{SBS}}(s)$ be the SBS action. We define

$$D(s) = \mathbf{1} [\exists a \in \mathcal{A}_6(s) : a \neq a_{\text{SBS}}(s)] . \quad (2)$$

The opportunity prevalence is the mean of Eq. 2, $P(D) = \sum_s D(s) / |\mathcal{S}_{\text{all}}|$, where the denominator is every valid decision state, not only states where policy scores differ. A policy ranking change that produces the same admission, batch, and request order is action collapse, not opportunity. An alternative action is any executable member of $\mathcal{A}_6(s)$ that is not equal to $a_{\text{SBS}}(s)$.

For every disagreement state s and canonical alternative a , we clone the state, force exactly one action, and return to the SBS for the continuation. Let $L_{\text{CF}}(s, a)$ be the mean latency of the resulting continuation and $L_{\text{SBS}}(s)$ that of the unmodified SBS continuation; both cover the active, queued, and future requests of the continuation and exclude requests completed before s . This is a local counterfactual, not selector performance: it estimates the one-step effect of changing the current action while holding the subsequent

policy fixed. We define

$$\begin{aligned} A_{\text{LAT}}(s, a) &= L_{\text{SBS}}(s) - L_{\text{CF}}(s, a), \\ H_{\text{LAT}}(s) &= \max \left(0, \max_a A_{\text{LAT}}(s, a) \right), \end{aligned} \tag{3}$$

where positive values reduce mean latency. The beneficial-state indicator is $B_{\text{LAT}}(s) = 1$ iff $\max_a A_{\text{LAT}}(s, a) > 0$. We report $P(B_{\text{LAT}} \mid D)$ and mean H_{LAT} separately from $P(D)$, so the conceptual aggregate opportunity is

$$P(D) \times P(B_{\text{LAT}} \mid D) \times \text{effect magnitude}, \tag{4}$$

not an exact deployable-performance equation. A high conditional benefit can coexist with a tiny overall opportunity. “Support” and “opportunity” refer only to the existence of executable non-SBS actions under a condition; usefulness requires the causal latency test above.

3.4. Endpoints and pre-specification

The primary endpoint is mean latency (Eq. 3). Deadline goodput is the natural objective for a deadline-constrained service, and we first measured it as arrival-normalized weighted goodput (ANWG), the weighted share of requests that complete by their SLO deadline. It cannot discriminate under the study’s loose deadlines: in a pilot causal analysis of the original windows every terminal ANWG value was 1.0, although most counterfactual branches changed latency. Latency was therefore chosen as the primary endpoint of the fresh study, and the pilot latency analysis is not used as confirmatory evidence.

The relative headroom $H_{\text{REL}}(s) = H_{\text{LAT}}(s)/L_{\text{SBS}}(s)$ is the oracle reduction as a share of the reference’s mean latency. The other secondary outcomes are the state structure (some alternative reduces latency, every alternative increases it, or neither) and the shares of alternative actions that reduce, increase, or leave latency unchanged. The pre-specified p95-latency effects are not reported here, and ANWG was not recorded in the fresh execution.

The confirmatory analysis was specified before any outcome was computed. Its protocol (primary endpoint, regime-selection rule, causal population, cluster unit, bootstrap settings, and decision rule) was committed to the project’s version-controlled repository on 19 September 2026, and the results were committed 2 h 14 min later. It was not deposited in an external registry, and the author controlled the repository, so we call the analysis

pre-specified rather than preregistered. The decision rule requires a positive state-weighted mean headroom and a window-clustered 95% lower endpoint above zero. One implementation error was corrected after the outcomes were known: the first bootstrap grouped windows by an index shared across workloads (26 clusters) instead of the specified source window (36 clusters). The point estimate and the decision were unchanged.

3.5. Statistical analysis

The confirmatory mean uses a window-clustered percentile bootstrap with 2,000 replicates and seed 20260920. The cluster unit is the fresh trace window, identified by workload source and window index, so multiple states from one window do not become independent experimental units. The analyses in Section 6 were specified after the fresh outcomes were known; they qualify the pre-specified result without changing its endpoint, strict criterion, 720 states, or selected regimes. A *window* is a workload source and window index (36 windows contribute states), and a window can contribute states to several regimes.

Practical thresholds. For $\epsilon \in \{0.1, 0.25, 0.5, 1, 2, 5\}$ ms we report the number and share of disagreement states with $H_{\text{LAT}}(s) > \epsilon$, that is $P(H_{\text{LAT}} > \epsilon \mid D)$, and the number of windows containing at least one such state.

Numerical residue. The pre-specified criterion is applied unchanged in double precision. Simulated latency differences are integer multiples of the decode step divided by the request population, so no genuine advantage is smaller than about 5.2×10^{-6} s; only to identify floating-point residue, we also count states whose best advantage exceeds 10^{-12} s.

Aggregation. For a unit u (a window, regime, or workload source) let D_u be its disagreement states and $\bar{H}_u = |D_u|^{-1} \sum_{s \in D_u} H_{\text{LAT}}(s)$. The pre-specified estimate is the *state-weighted mean* $\bar{H} = |D|^{-1} \sum_{s \in D} H_{\text{LAT}}(s)$, in which every state has weight $1/|D|$. The *equal-window*, *equal-regime*, and *equal-workload* means are $U^{-1} \sum_u \bar{H}_u$ over the 36 windows, five regimes, or two workloads, so every unit has weight $1/U$. These are different estimands, not alternative estimates of one quantity; the beneficial share is aggregated in the same way. Intervals use the same window-clustered percentile bootstrap with 200,000 replicates; for the equal-regime and equal-workload means, regimes and workloads are fixed strata, which ignores dependence between regimes that share a window.

Leave-one-out, concentration, and interval diagnostics. For each window (36) and each regime (5) we recompute the state-weighted mean and beneficial share from the remaining states with a window-clustered bootstrap interval (200,000 replicates). Concentration is the share of the total summed headroom $\sum_s H_{\text{LAT}}(s)$ contributed by each window and regime, and the effective number of windows $1/\sum_w p_w^2$, where p_w is the share of window w . To check the confirmatory interval we repeat the percentile bootstrap with 10^6 replicates and add a bias-corrected and accelerated (BCa) interval and a delete-one-window jackknife t interval; these are diagnostics, not replacements.

4. Where Disagreement Appears

4.1. Native replay

Under the abundant-resource baseline, all three production-derived workloads produced no canonical disagreement state (Figure 2). Azure code had 0 disagreements in 99,992 SBS decision states; Azure conversation had 0 in 461,985; and BurstGPT had 0 in 440,461. The corresponding maximum KV utilizations were 0.0027, 0.0017, and 0.0038, with no active-cap binding states, and the mean queue lengths were 0.040, 0.0087, and 0.0091 requests, so the replay was lightly loaded. Policy-ranking proxy scores differed in a small number of states, but the canonical actions collapsed to the SBS action. In this resource regime the six-policy portfolio offered no executable alternative.

4.2. Capacity pressure

The pressure map of the original windows evaluated 1,080 one-axis conditions: 20 windows per workload family, each under settings on the active-sequence, KV-capacity, and arrival-rate axes (Table 2). Of these, 926 conditions were valid and unconstrained, 38 partially constrained, 96 strongly constrained, and 20 horizon-invalid. Across valid conditions, 11,328 canonical disagreement states were observed.

Arrival scaling through $8\times$ produced no disagreement state in any of the three workload families. It also left the simulated system lightly loaded: peak KV utilization stayed below 0.71% of the default capacity, at most 46 sequences were active against a limit of 512, the mean queue length per workload stayed below 0.09, and no capacity constraint bound. Disagreement first appeared at an active-sequence cap of 16 (BurstGPT), 8 (Azure code),

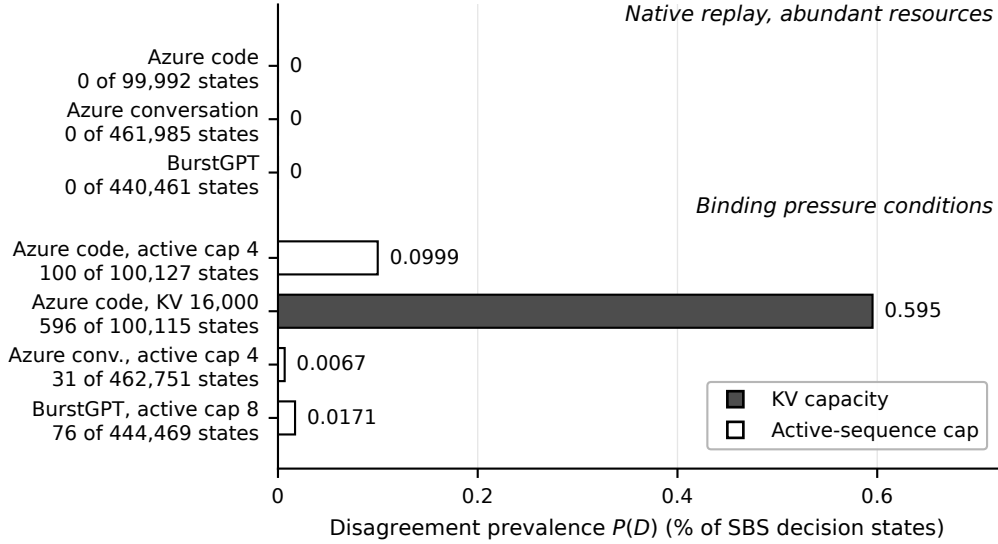


Figure 2: Disagreement prevalence $P(D)$, the percentage of SBS decision states that are disagreement states (counts in the axis labels), for native replay of the three workloads and for selected pressure conditions (conv. = conversation). Native replay produced no disagreement state. Fill denotes the pressure axis.

or 4 (Azure conversation), and at a KV capacity of 16,000 tokens in all three workloads (Table 2). At the Azure-code onset cap of 8, disagreement is a single state among 99,992 SBS decision states (0.001%), against 100 of 100,127 (0.0999%) at a cap of 4 (Figure 2). Disagreement became sustained, in at least two windows, at caps of 8 (BurstGPT) and 4 (Azure code and conversation) and at KV capacities of 16,000 (Azure code and BurstGPT) and 8,000 (Azure conversation).

Table 2 summarizes the regime result: in these replays, disagreement appeared only after an active-sequence or KV capacity was tightened. For Azure conversation under a KV cap, disagreement appears at 16,000 tokens, before the constraint physically binds at 8,000; Section 6.4 examines this pattern.

Table 2: Pressure settings at which canonical disagreement first appears. Settings are the active-sequence cap (concurrent sequences) and the KV capacity (tokens). “First binding” is the first setting at which the constraint binds; “sustained” is the first setting with disagreement in at least two windows.

Workload	Pressure axis	First binding	First disagreement	Sustained
Azure code	Active-sequence cap	8	8	4
Azure code	KV capacity	16,000	16,000	16,000
Azure conversation	Active-sequence cap	4	4	4
Azure conversation	KV capacity	8,000	16,000	8,000
BurstGPT	Active-sequence cap	16	16	8
BurstGPT	KV capacity	16,000	16,000	16,000

5. Causal Headroom on Fresh Windows

5.1. Support on fresh windows

The 1,080-condition pressure matrix was repeated on 60 fresh windows, 20 per workload, before any latency outcome was computed. The transition reproduced for both Azure traces: disagreement first appeared at a KV capacity of 16,000 tokens and at active-sequence caps of 8 (Azure code) and 4 (Azure conversation), the onset settings of Table 2, and arrival-rate scaling again produced none, with peak KV utilization below 0.45%. It did not reproduce for BurstGPT. All 360 fresh BurstGPT conditions were valid, none produced a disagreement state, and none had a binding capacity constraint: the fresh windows never queued more than three requests, so even the tightest caps (four active sequences, 8,000 KV tokens) were never reached, whereas native replay of the original BurstGPT windows reached a queue of 31 requests. The causal intervention covers five selected Azure regimes (active-sequence and KV-capacity caps) with 720 disagreement states in total.

5.2. Primary result

The confirmatory intervention found 590 beneficial states (Table 3; Figure 3), giving $P(B_{\text{LAT}} \mid D) = 590/720 = 0.8194$ (81.9%). Mean oracle latency headroom, in simulator time, was 0.001995791 s (1.9958 ms), with a clustered 95% confidence interval (CI) of [0.000190884, 0.003546196] s

([0.1909, 3.5462] ms) on the same simulated clock; none of these values is a measured hardware latency. The pre-specified decision rule was met. The conditional share is defined on disagreement states only; because $P(D)$ is below 0.5% in every selected regime (Table 3), it is not the share of all scheduler decisions that could be improved.

These state-weighted values are examined further in Section 6. One of the 590 beneficial states has an advantage of 5.6×10^{-17} s, exactly one unit in the last place of the 0.357 s latencies compared, with both branches completing the same requests; a post hoc guard of 10^{-12} s gives 589/720 (81.8%) and changes the mean by less than 10^{-19} s.

Table 3: Fresh causal characterization by selected regime, numbered as in Figure 3 (conv. = conversation; active cap = active-sequence cap; KV = KV capacity in tokens). $P(D)$ is the percentage of valid SBS decision states in the regime’s conditions that are disagreement states, $P(B_{\text{LAT}} | D)$ the percentage with positive oracle headroom, and mean H_{LAT} the state-weighted mean in simulated milliseconds (sim. ms). Windows can contribute to several regimes, so window counts are not additive; the 36 distinct workload–window pairs are the bootstrap clusters.

Regime	States	Windows	$P(D)$ (%)	$P(B_{\text{LAT}} D)$ (%)	Mean H_{LAT} (sim. ms)	Headroom share (%)
(1) Azure code, active cap 8	10	2	0.0106	40.0	0.033	0.02
(2) Azure code, active cap 4	122	15	0.129	52.5	0.077	0.65
(3) Azure code, KV 16,000	431	14	0.456	92.3	3.227	96.80
(4) Azure conv., active cap 4	64	17	0.0108	48.4	0.217	0.97
(5) Azure conv., KV 16,000	93	1	0.0157	100.0	0.241	1.56
All selected regimes	720	36	—	81.9	1.996	100.00

Figure 3 separates prevalence from value, two of the three factors of Eq. 4. Azure-code KV capacity = 16,000 has the largest selected $P(D)$ and the largest mean headroom; the active-cap regimes have much smaller $P(D)$ and correspondingly smaller aggregate opportunity even when some states are beneficial.

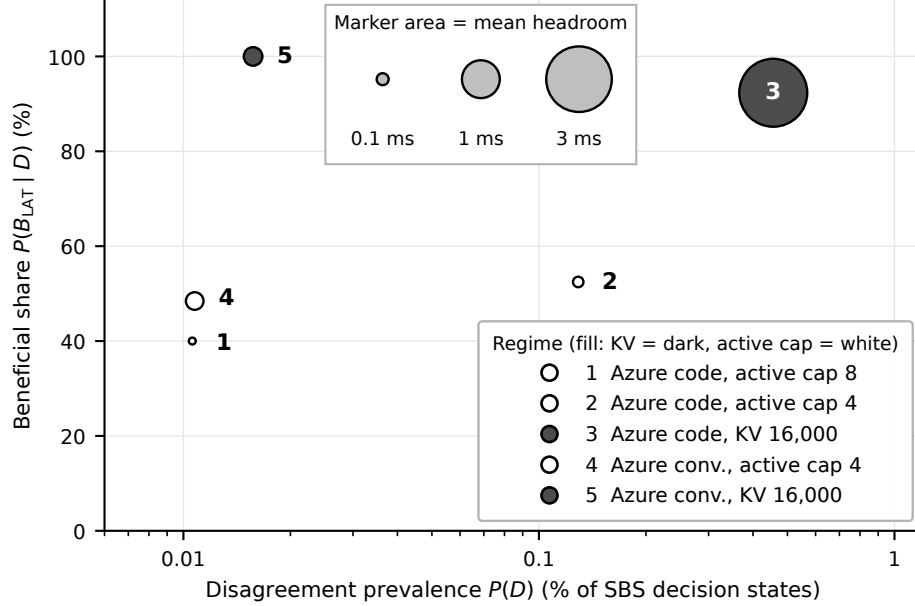


Figure 3: Regime map for the five selected regimes, numbered as in Table 3. Horizontal axis: disagreement prevalence $P(D)$ in percent (logarithmic scale). Vertical axis: beneficial share $P(B_{\text{LAT}} | D)$, which counts any positive headroom (regime 5, at 100%, has a mean headroom of 0.24 ms). Marker area is proportional to the mean oracle headroom; fill denotes the pressure axis.

5.3. Secondary outcomes

Table 4 reports the pre-specified secondary outcomes. In 102 of the 720 states (14.2%) every alternative increases latency, and 141 of the 831 alternative actions (17.0%) do so. The median relative headroom is 0.21%, but 92 states (12.8%) exceed 10% of the reference’s mean latency, and all 92 lie in the Azure-code KV-capacity = 16,000 regime, where they are 21.3% of the states.

6. Concentration, Sensitivity, and Reference Dependence

The pre-specified result of Section 5 is a state-weighted mean over 720 disagreement states drawn from 36 windows. This section examines, post hoc, how large the headroom is in a typical state, how far the mean depends on which windows and regimes contribute (Section 3.5), and how it depends on the reference policy.

Table 4: Pre-specified secondary outcomes over the 720 fresh disagreement states and their 831 alternative actions. Relative headroom is $H_{\text{LAT}}(s)/L_{\text{SBS}}(s)$, the oracle latency reduction as a share of the reference’s mean latency in the state. “Zero headroom” states have no beneficial alternative but not every alternative harmful.

Outcome	Result
<i>State structure (720 states)</i>	
Some alternative reduces latency	590 (81.9%)
of which some alternative also increases it	22
Every alternative increases latency	102 (14.2%)
Zero headroom, not every alternative harmful	28 (3.9%)
<i>Action level (831 alternative actions)</i>	
Reduce latency	660 (79.4%)
Increase latency	141 (17.0%)
Leave latency unchanged	30 (3.6%)
<i>Relative headroom H_{REL}</i>	
Median	0.21%
90th percentile	11.3%
90th percentile, Azure-code KV 16,000 only	16.1%
States above 1%	211 (29.3%)
States above 5%	133 (18.5%)
States above 10%	92 (12.8%)

6.1. Effect scale and distribution

Table 5 counts the states whose headroom exceeds each practical threshold. The 81.9% beneficial share is not the share of states that offer material gains: 60.0% of states exceed 0.1 ms, 22.5% exceed 2 ms, and 17.9% exceed 5 ms, and the larger effects are concentrated in one regime and a few windows (last two columns).

The distribution is strongly right-skewed (Figure 4a). The median headroom is 0.197 ms, about one tenth of the 1.996 ms state-weighted mean, and 77.4% of states lie at or below that mean. The 90th and 95th percentiles are 8.4 and 11.3 ms, and the maximum is 12.1 ms. A further 130 states (18.1%) have exactly zero headroom. Among the 589 states with positive headroom

Table 5: Post hoc practical-effect thresholds: disagreement states whose oracle headroom H_{LAT} exceeds each threshold. The first row is the pre-specified criterion; the second excludes one floating-point-residue state with a 10^{-12} s guard. “Windows” counts the windows (of 36) with at least one such state; “In KV 16,000” counts states from the Azure-code KV-capacity = 16,000 regime, which contributes 431 of the 720 states.

Criterion	States	Share	Windows	In KV 16,000
Pre-specified (> 0 , strict)	590	81.9%	32	398
Guarded ($> 10^{-12}$ s)	589	81.8%	32	398
> 0.1 ms	432	60.0%	17	327
> 0.25 ms	322	44.7%	16	259
> 0.5 ms	212	29.4%	9	201
> 1 ms	188	26.1%	7	185
> 2 ms	162	22.5%	4	161
> 5 ms	129	17.9%	2	128

the median is 0.29 ms.

6.2. Sensitivity to aggregation and to individual windows and regimes

Every aggregation of Table 6 yields a positive mean whose interval excludes zero, but the magnitude depends on what is weighted: from 1.996 ms (state-weighted) to 0.327 ms (equal-window), a factor of about six, while the beneficial share falls from 0.819 to 0.632.

Leaving one unit out gives the same picture. All 36 single-window and 5 single-regime omissions leave a positive mean, and every corresponding bootstrap interval excludes zero. Omitting Azure-code window w11 leaves 0.365 ms and omitting the Azure-code KV-capacity = 16,000 regime leaves 0.159 ms (Table 6); omitting any other window changes the mean only to between 2.00 and 2.26 ms, and omitting any other regime to between 2.02 and 2.39 ms. The sign therefore survives all 41 omissions, whereas the headline magnitude does not survive the omission of the two dominant units.

6.3. Concentration of headroom

Window w11 of the Azure-code trace contains 263 of the 720 states (36.5%) but contributes 88.4% of the total summed headroom; the three largest windows contribute 95.3%, and the effective number of windows by headroom mass is 1.27 (5.7 by state count). At regime level, Azure-code

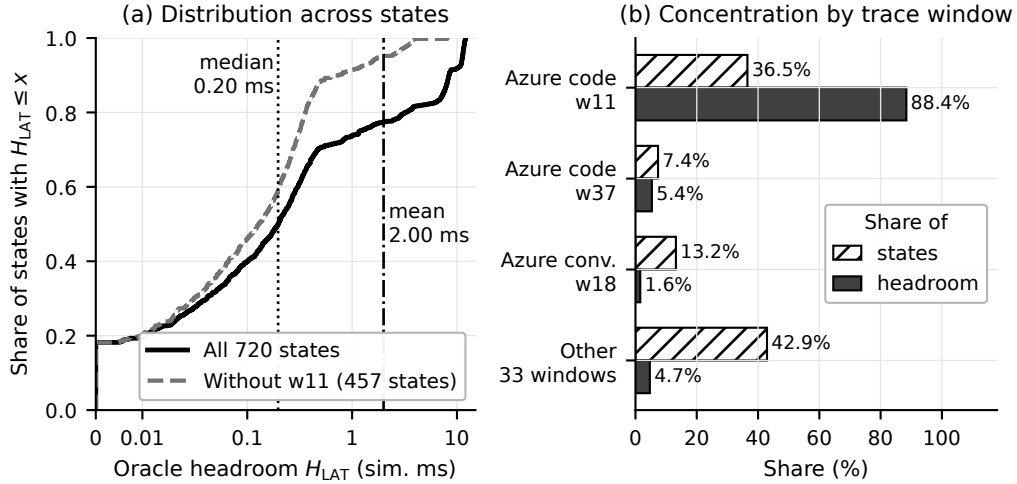


Figure 4: Distribution and concentration of oracle headroom over the 720 disagreement states. (a) Empirical distribution of H_{LAT} with and without Azure-code window w11; zero headroom is the intercept at $x = 0$, and the axis is linear below 0.01 ms and logarithmic above. Dotted and dash-dotted lines mark the median and the state-weighted mean. (b) Share of the 720 states (hatched) and of the total headroom (solid) for the three largest windows and the remaining 33.

Table 6: Post hoc sensitivity of the mean oracle headroom to the aggregation rule and to leaving out the two dominant units. Each unit has equal weight except in the state-weighted row, where each state does. “Share” is the beneficial share under the strict > 0 criterion. The state-weighted interval is the confirmatory one (2,000 replicates); the others are window-clustered percentile bootstrap intervals with 200,000 replicates (Section 3.5).

Estimand	Units	Mean (sim. ms)	95% CI (sim. ms)	Share
State-weighted (pre-specified)	720 states	1.996	[0.191, 3.546]	0.819
Equal-workload	2 workloads	1.360	[0.184, 2.094]	0.809
Equal-regime	5 regimes	0.759	[0.145, 1.161]	0.666
Equal-window	36 windows	0.327	[0.114, 0.644]	0.632
<i>Leave one out: state-weighted mean of the remaining states</i>				
Omit Azure-code window w11	457 states	0.365	[0.158, 0.682]	0.821
Omit Azure-code KV 16,000	289 states	0.159	[0.058, 0.234]	0.664

KV-capacity = 16,000 holds 59.9% of the states and 96.8% of the headroom, contributing 1.93 of the 2.00 ms state-weighted mean. Within w11, 205 states of that regime average 6.16 ms. Of the 129 states above 5 ms, 128 are in w11, as are 140 of the 162 states above 2 ms (Figure 4b). The state-weighted mean is thus highly concentrated in a narrow high-headroom subset of states. Appendix A shows that the width of the confirmatory interval reflects this concentration, not simulation noise.

6.4. *Dependence on the reference scheduler*

The SBS is a fixed heuristic, and the headroom is measured against it. In 512 of the 720 disagreement states (71%) all five other policies differ from the SBS, and these states carry 94.8% of the total headroom; 511 of them lie in the two KV-capacity regimes, and in 445 the five policies agree on a single alternative action. The disagreement is therefore mainly between the reference and the rest of the portfolio, not among the alternatives. The KV-capacity regimes contain 524 disagreement states, of which 414 occur without physical KV binding (the waiting prompt tokens fit in the free capacity) and hold 89.6% of those regimes’ headroom. These are valid observations: the reference reorders or withholds admissions there although capacity is not exhausted, so physical exhaustion does not explain them. The reference’s 0.82 reserve and its KV-footprint ordering, which no other portfolio policy uses, are the candidate sources, and this analysis does not separate the two.

7. A Bounded vLLM System Correspondence Probe

The probe checks whether the mechanism found in simulation, that resource pressure changes queueing and scheduling behavior, also appears in a real engine; it is a correspondence probe, not a replication of simulated magnitudes. It used one RTX 5060 Ti GPU, vLLM 0.27.1, Qwen2.5-0.5B-Instruct, and deterministic token-shape replay of requests shaped after the Azure workloads. It comprised 40 full-versus-chunked runs and a 20-run native chunked-prefill budget probe, with queueing and scheduler traces retained. Three levels of evidence are distinguished. Level 1, one-step headroom at disagreement states, is established only in the controlled replay. Level 2, that resource pressure changes observable queueing and scheduling behavior, is supported in vLLM. Level 3, that a deployed adaptive selector would recover the simulated headroom or improve end-to-end latency, SLOs, or throughput, is not established.

The native budget probe held chunked prefill fixed and varied the per-step token budget (512 tokens, T512, versus 4,096 tokens, T4096). T512 produced 1,680 scheduler steps, 165 mixed prefill/decode steps, and 194 partial-prefill items; T4096 produced 1,536, 55, and 9. Waiting queues reached lengths of 7 and 5, respectively, while running sequences reached 4 in both cases. In measured vLLM latency, the low-load regime showed a 30.6 ms difference in late time to first token and a 16.3 ms prompt-heavy end-to-end tradeoff; the high-load regime showed a smaller time-to-first-token difference and a 23.3 ms prompt-heavy end-to-end tradeoff. These are whole-run effects of changing the token budget, not one-step causal effects, and the scheduler-step counts are the nearest observable signature of an action difference, not an exact counterpart of the simulator’s canonical action.

The direct full-versus-chunked comparison used workloads in which long-prompt requests arrive first and short requests arrive later. The simulator predicted a reversal between the two classes: chunked prefill favoring the late short requests and full prefill favoring the long-prompt requests. vLLM did not reproduce this reversal: chunked prefill did not lower late-request time to first token in the late-tight regimes, and full prefill showed no stable end-to-end advantage for the long-prompt requests. The probe therefore shows that concrete serving pressure alters observable scheduling behavior and latency; it does not estimate Eq. 3 in the real system, because an exact one-step intervention is not available in the engine.

8. Discussion

8.1. *Sign and magnitude of the headroom*

Forcing an alternative action lowers mean latency in most fresh disagreement states (590 of 720). The sign is stable across the aggregation and omission analyses of Section 6; the magnitude is not. The state-weighted mean estimates the expected headroom at a state drawn from this sample, not the headroom of a typical state: the median is 0.20 ms, and the mean falls to 0.33 ms when windows receive equal weight (Table 6). “Beneficial” is a criterion on the sign of the causal effect, not on its operational size: many states offer little, in 14.2% every alternative is worse than the reference, and one window carries 88.4% of the total headroom (Table 4, Figure 4). An aggregate mean can therefore conceal highly localized scheduling opportunity, so the distribution of effects, the prevalence of material effects, and the sensitivity to aggregation belong next to any overall mean.

8.2. Opportunity, prediction, and deployment

Adaptive scheduling should be evaluated in stages, because each stage can fail while the previous one holds. Resource pressure need not produce executable disagreement, disagreement need not be beneficial, a positive effect need not be material, and a material oracle effect need not be recoverable. The estimand is one-step oracle headroom under SBS continuation: $H_{\text{LAT}}(s)$ selects the best alternative after the outcomes are observed, so it answers whether a useful action existed at s , not whether an online system could have identified it, and it does not measure a selector, longer-horizon switching, or adaptation overhead.

Each stage is also measured against one fixed reference and one portfolio. Because about 95% of the headroom lies where all five other policies differ from the reference, and most KV-regime disagreement occurs without physical KV binding (Section 6.4), the measurement is largely the room to improve on this reference’s reserve and ordering under a capped KV capacity. It is neither evidence that the portfolio is complementary nor a general property of KV-limited serving. Score-level complementarity is a separate question: on the 60 native windows, small-chunk prefill has a higher whole-window mean latency than full prefill in all 60 (median 14.2%), yet the two issue the same action from any state and differ only in execution chunk size, so no per-state choice can realize that gap. The data illustrate the possibility without measuring a six-policy gap, which was not evaluated in the faithful view.

8.3. Resource pressure and action support

Caps on active sequences and KV capacity created disagreement in all three workloads on the original windows (Section 4), whereas arrival-rate scaling up to eight times created none. This is a negative result under light load: it does not show that arrival intensity cannot create opportunity in a congested system. In these production-derived workloads under controlled resource interventions, disagreement appeared only after a capacity setting was tightened, and where a trace enters that regime depends on its windows: BurstGPT disagreement appeared in the original windows but not in the fresh ones, so the onset settings of Table 2 are not workload constants. The vLLM experiment is a bounded system correspondence probe (Section 7): it supports the qualitative link between pressure and scheduling behavior, did not reproduce the simulator’s predicted class reversal in the direct full-

versus-chunked comparison, and does not validate the simulated headroom magnitude or an adaptive-scheduler gain.

8.4. *Implications for practice*

The evidence supports a measurement-first practice, not a scheduler recommendation.

- Measure executable action disagreement before building a selector; diversity in policy scores is not evidence of it.
- Locate the operating regions where scheduler choice matters: characterize which resource constraints bind, in addition to the arrival rate, and state the reference policy and how it was chosen.
- Report the distribution of effects in absolute and relative terms, their concentration, and the share of states in which every alternative is harmful, alongside any oracle mean.
- Weigh the recoverable gain against selector, monitoring, and switching costs; outside the few regimes with material opportunity, a fixed scheduler may be the appropriate policy.

The study demonstrates no production savings, deployed-selector benefit, capacity-planning gain, or universal scheduling recommendation.

8.5. *Limitations and threats to validity*

The findings are conditional on the following boundaries.

Reference and portfolio. The SBS is a fixed heuristic chosen on a separate benchmark, and its hard-coded 0.82 KV reserve partly explains the measured headroom (Section 6.4). Conclusions are conditional on it, and a differently tuned reference could shrink or shift the opportunity. The six policies are simple heuristics, two of which coincide in action and two of which reduce to one ranking under the overlays, so other portfolios may expose different action support. Four policies use exact output lengths, so length-aware rules face no prediction error, and constant priorities and 1000 s deadlines make deadline- and priority-aware rules uninformative; the results describe this idealized information setting.

Load and workloads. Active-sequence and KV caps are experimental interventions, not observed production provisioning, and arrival-rate scaling up

to eight times left the simulated system lightly loaded, so the arrival results do not bound the effect of higher congestion. The fresh causal study covers Azure code and conversation only, because the fresh BurstGPT windows never queued enough for the caps to bind, and fresh windows were taken in trace order, not sampled at random. The simulator is uncalibrated (a 1 ms step), so absolute milliseconds are simulator units, and other models, hardware, schedulers, and context-length distributions may show a different opportunity structure. The staged protocol transfers; the measured magnitudes need not.

Estimand and inference. H_{LAT} isolates one-step causal opportunity with an ex post best alternative; it is not closed-loop adaptation. The magnitude rests on few windows (an effective number of about 1.27 by headroom mass), whereas the sign has broader support: 32 of the 36 windows contain a state with positive headroom. Mean latency is the primary endpoint, and tail-latency and SLO-violation objectives were not evaluated.

System correspondence. The probe uses one model, one GPU, one vLLM version, and token-shape replay, and KV utilization and preemption could not be collected in its native budget probe, so it supports only the qualitative mechanism.

9. Conclusion

This paper asked whether an LLM-serving scheduler ever faces an executable alternative worth taking before any adaptive scheduler is built. In production-derived replay with abundant resources it did not: none of about one million decision states offered an action different from the single best scheduler (SBS), a fixed reference. Capacity caps on active sequences or KV cache created such states, whereas lightly loaded arrival-rate scaling did not. On the Azure traces, forcing an alternative once lowered latency in most of these states (590 of 720 under the pre-specified criterion).

The opportunity is nonetheless small, concentrated, and conditional. The state-weighted mean simulated headroom of 2.00 ms is not a typical effect: the median disagreement state offers 0.20 ms, and one trace window holds 88.4% of the total headroom. Most of the headroom lies where all five other policies differ from the reference, so it measures room to improve on that reference rather than a general property of LLM serving.

The lesson for performance evaluation is to establish, stage by stage, that executable disagreement, causal benefit, and a material effect exist before

adding selector complexity, and to report the reference, the portfolio, the load range, and the distribution of effects with any measured opportunity. Adaptation is justified by where the measured opportunity lies, not by its average.

Appendix A. Interval Diagnostics

Monte Carlo error does not explain the width of the confirmatory interval. With 10^6 replicates the percentile interval is $[0.184, 3.543]$ ms, no replicate has a nonpositive mean, and the lower endpoint of the 2,000-replicate procedure varies by only 0.004 ms (standard deviation) across 1000 seeds. The BCa interval is $[0.239, 4.049]$ ms. The bootstrap distribution is bimodal: 36% of replicates do not draw window w11 and have a mean of about 0.36 ms, against 2.43 ms for those that do, so the percentile interval spans the gap between two populations of replicates. The delete-one-window jackknife t interval, $[-1.36, 5.35]$ ms, includes zero because one influential window inflates its standard error (1.65 ms); all 36 delete-one estimates are positive, and we treat the jackknife interval as a diagnostic of this dependence rather than as a competing interval. Uncertainty about the magnitude of the headroom thus reflects its concentration in few windows, not simulation noise.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the author used ChatGPT (OpenAI), Codex (OpenAI), Gemini (Google), and Claude (Anthropic) in order to assist with organization, language editing, literature-search planning, and drafting analysis, figure, and test code. After using these tools, the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

CRedit authorship contribution statement

Soroush Vahidi: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Visualization, Writing – original draft, Writing – review & editing.

Declaration of Competing Interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding and Support

This work received in-kind computational support through the Google Cloud Research Credits Program (USD 1,000 in cloud credits) and computational/tooling support from CloudRift Inc. Neither Google nor CloudRift Inc. had any role in the study design, data collection, analysis, interpretation, manuscript preparation decisions, or the decision to submit the article for publication. No monetary research grant was received from these organizations.

Data and Code Availability

Code, derived research artifacts, experiment configurations, and reproducibility materials for this study are available in the GitHub repository <https://github.com/SoroushVahidi/llm-serving-heuristic-evolution> and are archived on Zenodo [38]. The v1.1.0 manuscript release package archives the confirmatory artifacts, robustness outputs, continuation shards, figure code, claim manifest, simulator, and analysis code. Raw third-party workload traces are not redistributed and are obtained from their original upstream sources.

Acknowledgements

The author gratefully acknowledges Professor Ioannis Koutis for research guidance and scientific feedback. The author thanks the NJIT Wulver HPC for providing computational resources for the fresh confirmatory computation, and Anders Borum (Secure ShellFish) for providing tools that enabled remote research access. Finally, the author thanks the author’s mother for continued support.

References

- [1] J. R. Rice, The algorithm selection problem, *Advances in Computers* 15 (1976) 65–118.
- [2] C. P. Gomes, B. Selman, Algorithm portfolios, *Artificial Intelligence* 126 (1–2) (2001) 43–62.
- [3] B. Sun, Z. Huang, H. Zhao, W. Xiao, X. Zhang, Y. Li, W. Lin, Llumnix: Dynamic scheduling for large language model serving, in: 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24), USENIX Association, 2024, pp. 173–191.
- [4] V. Srivatsa, Z. He, R. Abhyankar, D. Li, Y. Zhang, Preble: Efficient distributed prompt scheduling for LLM serving, in: The Thirteenth International Conference on Learning Representations (ICLR 2025), 2025.
- [5] D. Zhang, J. Han, K. Zhang, X. Wei, S. Shen, C. Fang, W. Yu, J. Zhou, R. Chen, Simple is better: Multiplication may be all you need for LLM request scheduling, in: 20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26), USENIX Association, 2026, pp. 55–73.
- [6] K. Hong, X. Li, L. Chen, Q. Mao, G. Dai, X. Ning, S. Yan, Y. Liang, Y. Wang, SOLA: Optimizing SLO attainment for large language model serving with state-aware scheduling, in: *Proceedings of Machine Learning and Systems*, Vol. 7, 2025.
- [7] W. Zhang, Z. Wu, Y. Mu, R. Ning, B. Liu, N. Sarda, M. Lee, F. Lai, JITServe: SLO-aware LLM serving with imprecise request information, in: 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26), USENIX Association, 2026, pp. 825–848.
- [8] Z. Li, Z. Chen, R. Delacourt, G. Oliaro, Z. Wang, Q. Chen, S. Lin, A. Yang, Z. Zhang, Z. Chen, Y.-H. Lai, X. Cheng, X. Miao, Z. Jia, AdaServe: Accelerating multi-SLO LLM serving with SLO-customized speculative decoding, in: *Proceedings of the 21st European Conference on Computer Systems (EuroSys 26)*, ACM, 2026. doi:10.1145/3767295.3769315.

- [9] Microsoft Azure, Azure LLM inference trace 2023, <https://github.com/Azure/AzurePublicDataset/blob/master/AzureLLMInferenceDataset2023.md>, public LLM inference trace dataset released by Azure (2023).
- [10] Y. Wang, Y. Chen, Z. Li, X. Kang, Y. Fang, Y. Zhou, Y. Zheng, Z. Tang, X. He, R. Guo, X. Wang, Q. Wang, A. C. Zhou, X. Chu, BurstGPT: A real-world workload dataset to optimize LLM serving systems, in: Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining, ACM, 2025, pp. 5831–5841. doi:10.1145/3711896.3737413.
- [11] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, I. Stoica, Efficient memory management for large language model serving with PagedAttention, in: Proceedings of the 29th Symposium on Operating Systems Principles (SOSP 2023), ACM, 2023, pp. 611–626.
- [12] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, B.-G. Chun, Orca: A distributed serving system for transformer-based generative models, in: 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22), USENIX Association, 2022, pp. 521–538.
- [13] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, R. Ramjee, Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve, in: 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24), USENIX Association, 2024, pp. 117–134.
- [14] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, H. Zhang, DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving, in: 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24), USENIX Association, 2024, pp. 193–210.
- [15] P. Patel, E. Choukse, C. Zhang, A. Shah, I. Goiri, S. Maleki, R. Bianchini, Splitwise: Efficient generative LLM inference using phase splitting, in: Proceedings of the 51st Annual International Symposium on Computer Architecture, IEEE Computer Society, 2024. doi:10.1109/ISCA59077.2024.00019.

- [16] B. Wu, Y. Zhong, Z. Zhang, S. Liu, F. Liu, Y. Sun, G. Huang, X. Liu, X. Jin, FastServe: Iteration-Level preemptive scheduling for large language model inference, in: 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26), USENIX Association, 2026, pp. 57–74.
- [17] C. Ruan, Y. Chen, D. Tian, Y. Shi, Y. Wu, J. Li, C. Li, Libra: Flexible request partitioning and scheduling for serving unbalanced and dynamic LLM workloads, in: 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26), USENIX Association, 2026, pp. 1243–1258.
- [18] R. Qin, Z. Li, W. He, J. Cui, F. Ren, M. Zhang, Y. Wu, W. Zheng, X. Xu, Mooncake: Trading more storage for less computation – a KVCache-centric architecture for serving LLM chatbot, in: 23rd USENIX Conference on File and Storage Technologies (FAST 25), USENIX Association, 2025, pp. 155–170.
- [19] Z. Xie, Z. Xu, M. Zhao, Y. An, V. S. Mailthody, S. Mahlke, M. Garland, C. Kozyrakis, Strata: Hierarchical context caching for long context language model serving, in: 20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26), USENIX Association, 2026, pp. 1–16.
- [20] Z. Su, Z. Zhang, T. Lan, Z. Wang, H. Shen, J. Yang, Y. Cheng, MorphServe: Efficient and workload-aware LLM serving via runtime quantized layer swapping and KV cache resizing, in: Proceedings of Machine Learning and Systems, Vol. 8, 2026.
- [21] S. Gao, X. Zhang, Y. Shen, L. Chen, Apt-Serve: Adaptive request scheduling on hybrid cache for scalable LLM inference serving, Proceedings of the ACM on Management of Data 3 (3) (2025) 1–28. doi:10.1145/3725394.
- [22] J. Ye, A. Maurya, K. T. Chitty-Venkata, B. Nicolae, A. Kougkas, X.-H. Sun, PKAS: Predictive KVCache-Aware scheduling for faster LLM and transformer inferences, in: Proceedings of the 35th International Symposium on High-Performance Parallel and Distributed Computing (HPDC 26), ACM, 2026, pp. 402–414. doi:10.1145/3806645.3807819.

- [23] T. Sämann, P-PAS: Prefill-pressure adaptive scheduling for long-context LLM serving, arXiv preprint arXiv:2608.15171 (2026).
- [24] J. Stojkovic, C. Zhang, I. Goiri, J. Torrellas, E. Choukse, DynamoLLM: Designing LLM inference clusters for performance and energy efficiency, in: Proceedings of the 2025 IEEE International Symposium on High Performance Computer Architecture (HPCA 25), IEEE, 2025, pp. 1348–1362. doi:10.1109/HPCA61900.2025.00102.
- [25] X. Yao, Y. Jiang, I. Badanin, Q. Hu, R. M. Smith, B. Yuan, I. Schlag, E. Yoneki, A. Klimovic, OpenTela: Unifying decentralized computing resources for heterogeneous LLM serving, in: 20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26), USENIX Association, 2026, pp. 1821–1838.
- [26] Y. Xiang, X. Li, K. Qian, Y. Zhang, W. Yu, E. Zhai, X. Jin, J. Zhou, ServeGen: Workload characterization and generation of large language model serving in production, in: 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26), USENIX Association, 2026, pp. 1845–1859.
- [27] W. Nixon, J. Durbin, F. Standhartinger, H. S. Gunawi, J. Yang, A year in LLM serving: Workload evolution, caching and load-balancing, arXiv preprint arXiv:2608.13573 (2026).
- [28] P. Jaillet, J. Jiang, K. Mellou, M. Molinaro, C. Podimata, Z. Zhou, Online scheduling for LLM inference with KV cache constraints, arXiv preprint arXiv:2502.07115 (2025).
- [29] A. Bari, P. Hegde, G. de Veciana, Optimal scheduling algorithms for LLM inference: Theory and practice, Proceedings of the ACM on Measurement and Analysis of Computing Systems 9 (3) (2025) 1–43. doi:10.1145/3771574.
- [30] M. Mitzenmacher, R. Shahout, Queueing, predictions, and large language models: Challenges and open problems, Stochastic Systems 15 (3) (2025) 195–219. doi:10.1287/stsy.2025.0106.
- [31] Y. Li, Y. Chen, J. Chen, E. Choukse, H. Qiu, G. E. Suh, R. Fonseca, Z. Scully, U. Gupta, Beyond prediction: Tail-aware scheduling for LLM

- inference, in: Proceedings of the 43rd International Conference on Machine Learning (ICML 2026), 2026, also available as arXiv:2606.18431.
- [32] A. Agrawal, N. Kedia, J. Mohan, A. Panwar, N. Kwatra, B. S. Gulavani, R. Ramjee, A. Tumanov, VIDUR: A large-scale simulation framework for LLM inference, in: Proceedings of Machine Learning and Systems, Vol. 6, 2024.
 - [33] T. Choudhury, G. Joshi, W. Wang, Job assignment in machine learning inference systems with accuracy constraints, *Performance Evaluation* 167 (2025) 102463. doi:10.1016/j.peva.2024.102463.
 - [34] M. Yildiz, A. Rolich, A. Baiocchi, Dispatching policies in data center clusters: Insights from Google and Alibaba workloads, *Performance Evaluation* 172 (2026) 102551. doi:10.1016/j.peva.2026.102551.
 - [35] Q. Liang, H. Wang, P. Shenoy, Lilou: Resource-aware model-driven latency prediction for GPU-accelerated model serving, *Performance Evaluation* 171 (2026) 102539. doi:10.1016/j.peva.2025.102539.
 - [36] L. Xu, F. Hutter, H. H. Hoos, K. Leyton-Brown, SATzilla: Portfolio-based algorithm selection for SAT, *Journal of Artificial Intelligence Research* 32 (2008) 565–606.
 - [37] M. Lindauer, H. H. Hoos, F. Hutter, T. Schaub, AutoFolio: An automatically configured algorithm selector, *Journal of Artificial Intelligence Research* 53 (2015) 745–778.
 - [38] [dataset] S. Vahidi, Reproducibility package for “When Does LLM-Serving Scheduler Adaptation Matter? Action Opportunity and Causal Headroom in Production-Derived Replay”: data, code and manuscript sources, Zenodo dataset and software archive, published v1.1.0 record: <https://doi.org/10.5281/zenodo.22866983> (concept DOI: 10.5281/zenodo.22865293) (2026).